Lección 1 del curso - Vídeo 05: las propiedades de los números reales, con regla y compás

Marcos Bujosa

13 de octubre de 2026

Índice

1. Las propiedades de los números reales, con regla y compás - V05 **2**
1.1. Guion (escenas) . . . 2
1.2. Ensamblado de los ficheros de python (uno por idioma) . . . 2
1.3. Clases de las escenas . . . 3
1.4. Título del vídeo . . . 5
1.5. Código de cada escena . . . 5
1.5.1. Portada (escena 0) . . . 5
1.5.2. Escena 1 - Introducción . . . 5
1.5.3. Escena 2 - La suma, trasladando segmentos . . . 7
1.5.4. Escena 3 - El producto: la figura del vídeo 4 es una homotecia . . . 13
1.5.5. Escena 4 - Conmutativa y asociativa del producto: se puede, pero es engorroso . . . 18
1.5.6. Escena 5 - Las nueve reglas y «la notación lo es todo» 21

2. Trozos comunes de código **23**
2.1. Carga de las librerías Manim, $\text{manim}_{\text{voiceover}}$ y NacAL . . . 23
2.2. Idioma del vídeo . . . 24
2.3. Utilidades bilingües . . . 24
2.4. Código de colores de la serie . . . 24
2.5. Uso del paquete de LaTeX con la notación del curso (`nacal.sty`) 25

3. Rodando: 1,2,3... ¡acción! **29**
3.1. Generamos un fichero `mpeg` por cada escena . . . 29

3.2. Montaje: concatenar las escenas, música de entrada y salida, subtítulos 29
3.3. Comprobar las uniones y los subtítulos 32
3.4. Extraer el guion (la narración) para revisarlo 32
3.5. Tangle desde la terminal (sin abrir Emacs) 34

1. Las propiedades de los números reales, con regla y compás - V05

Vídeo que no sigue ninguna sección del libro: muestra geométricamente (regla y compás) las ocho propiedades algebraicas de los números reales que la Sección 1.2.1 da por conocidas. Tesis: se puede construir todo, pero es incómodo, y de esa incomodidad nacen las reglas de manipulación simbólica (prefacio del libro: "Notation is everything"). El vídeo ilustra, no demuestra; se dice explícitamente.

Este fichero es una **fuente única bilingüe** (véase la plantilla `LXX-VYY-NombreVideo.org`): genera `./lib/L01_V05_PropiedadesDeLosRealesConReglaYCompas.py` (español) y `../videoMath2/lib/L01_V05_RealNumberPropertiesWithRulerAndCompass.py` (inglés).

1.1. Guion (escenas)

Escena	Contenido
0	Portada (sobre ella suena la música)
1	Introducción: los reales como puntos de una recta; veremos, no demostraremos
2	La suma poniendo el origen de un segmento en el extremo de otro: conmutativa, asociativa
3	El producto con la figura del vídeo 4 (homotecia): el uno, el inverso y la distributiva
4	Conmutativa y asociativa del producto: se puede, pero es engorroso; Euclides y Descartes
5	Las nueve reglas, "la notación lo es todo", anuncio del siguiente vídeo

1.2. Ensamblado de los ficheros de python (uno por idioma)

```
<<Carga de la librería Manim y NacAL>>

<<Idioma es>>

<<Utilidades bilingües>>

<<Código de colores>>
```

```
<<Título del vídeo>>

<<TexTemplate con LaTeX-nacal>>

<<Definición de mi escena con voz>>

<<Clases de las escenas>>
```

Figura 1: Fichero de python del vídeo en español

```
<<Carga de la librería Manim y NacAL>>

<<Idioma en>>

<<Utilidades bilingües>>

<<Código de colores>>

<<Título del vídeo>>

<<TexTemplate con LaTeX-nacal>>

<<Definición de mi escena con voz>>

<<Clases de las escenas>>
```

Figura 2: Fichero de python del vídeo en inglés

1.3. Clases de las escenas

```
class L01_V05_PropiedadesDeLosRealesConReglaYCompas(MiEscenaConVoz):
    def construct(self):
        <<Portada>>

        self.play(*[FadeOut(mob) for mob in self.mobjects])  # borrado de todo

        <<Primera escena>>
```

```
        self.play(*[FadeOut(mob) for mob in self.mobjects])  # borrado de todo

        <<Segunda escena>>

        self.play(*[FadeOut(mob) for mob in self.mobjects])  # borrado de todo

        <<Tercera escena>>

        self.play(*[FadeOut(mob) for mob in self.mobjects])  # borrado de todo

        <<Cuarta escena>>

        self.play(*[FadeOut(mob) for mob in self.mobjects])  # borrado de todo

        <<Quinta escena>>

        self.play(*[FadeOut(mob) for mob in self.mobjects])  # borrado de todo

        <<Créditos finales>>


class L01_V05_E00_Portada(MiEscenaConVoz):
    def construct(self):
        <<Portada>>


class L01_V05_E01_Introduccion(MiEscenaConVoz):
    def construct(self):
        <<Primera escena>>


class L01_V05_E02_Suma(MiEscenaConVoz):
    def construct(self):
        <<Segunda escena>>


class L01_V05_E03_Producto(MiEscenaConVoz):
    def construct(self):
        <<Tercera escena>>
```

```
class L01_V05_E04_ProductoIncomodo(MiEscenaConVoz):
    def construct(self):
        <<Cuarta escena>>


class L01_V05_E05_Resumen(MiEscenaConVoz):
    def construct(self):
        <<Quinta escena>>


class ZCreditosFinales(Scene):
    def construct(self):
        <<Créditos finales>>
```

Figura 3: Vídeo completo y escenas separadas

1.4. Título del vídeo

```
TITULO = tr("Las propiedades de los números reales, con regla y compás",
            "The properties of the real numbers, with ruler and compass")
```

1.5. Código de cada escena

Cada escena es un chunk con nombre. Dentro, cada frase de la narración va junto a la animación que acompaña, y se escribe en los dos idiomas con `tr(es, en)`.

1.5.1. Portada (escena 0)

```
self.creditos()
portada = Tex(TITULO, tex_template=myTemplate, font_size=60).set_color(COLOR_TITULO)
portada.scale_to_fit_width(min(portada.width, config.frame_width*0.9))
self.play(FadeIn(portada, shift=0.3*UP), run_time=1.5)
self.wait(4.5)
```

1.5.2. Escena 1 - Introducción

Por qué este vídeo: para estudiar las operaciones con vectores hacen falta las propiedades algebraicas de los reales. Las vamos a ver (no demostrar) con regla y compás, y veremos que es incómodo.

```
self.creditos()
titulo = Title(TITULO, tex_template=myTemplate, font_size=44).set_color(COLOR_TITULO)
self.play(Write(titulo))

# las dos definiciones (vídeos 3 y 4), con los glifos coloreados según el código de l
f_suma = MathTex(r"\elemRp{\Vect{a}+\Vect{b}}{i}", "=", r"\eleVR{a}{i}+\eleVR{b}{i}",
                 tex_template=myTemplate, font_size=56).shift(0.9*UP)
f_prod = MathTex(r"\elemRp{\lambda\Vect{a}}{i}", "=", r"\lambda\eleVRpE{a}{i}",
                 tex_template=myTemplate, font_size=56).shift(0.5*DOWN)
for g in (f_suma[0][1], f_suma[2][0], f_prod[0][2], f_prod[2][2]):
    g.set_color(COLOR_A)
for g in (f_suma[0][3], f_suma[2][4]):
    g.set_color(COLOR_B)
for g in (f_prod[0][1], f_prod[2][0]):
    g.set_color(COLOR_ESCALAR)
indices = [f_suma[0][6], f_suma[2][2], f_suma[2][6], f_prod[0][5], f_prod[2][4]]

with self.voiceover(text=tr("""En los vídeos anteriores definimos dos operaciones con
                            """In the previous videos we defined two operations with
wise, that is, <bookmark mark='reales'/>by adding and multiplying real numbers.""")):
    self.wait_until_bookmark("suma")
    self.play(FadeIn(f_suma), run_time=1)
    self.wait_until_bookmark("producto")
    self.play(FadeIn(f_prod), run_time=1)
    self.wait_until_bookmark("componentes")
    self.play(*[Indicate(g, color=COLOR_ESCALAR, scale_factor=1.8) for g in indices],
    self.wait_until_bookmark("reales")
    self.play(Circumscribe(f_suma[2], color=COLOR_ESCALAR, buff=0.15),
              Circumscribe(f_prod[2], color=COLOR_ESCALAR, buff=0.15), run_time=2)

with self.voiceover(text=tr("""El siguiente paso es estudiar las propiedades que tien
                            """The next step is to study the properties of those two
    ocho = Tex(tr("Propiedades algebraicas de los números reales", "Algebraic propert
               font_size=48).set_color(COLOR_REGLA)
    self.pausa(tracker.duration/3)
    self.play(FadeOut(f_suma, f_prod), FadeIn(ocho), run_time=tracker.duration/3)

# recta con regla y compás
recta = NumberLine(x_range=[-1, 7, 1], length=10, include_numbers=False).shift(1.2*DO
recta.add_numbers([0, 1], font_size=36)
```

```
u = recta.n2p(1)[0] - recta.n2p(0)[0]
pa = recta.n2p(2.6)
punto_a = Dot(pa, color=COLOR_A, radius=0.1)
etq_a = MathTex("a", color=COLOR_A, font_size=46).next_to(punto_a, UP, buff=0.25)
arco = Arc(radius=2.6*u, start_angle=PI/2, angle=-PI/2, arc_center=recta.n2p(0),
           color=GRAY_B, stroke_width=2)

with self.voiceover(text=tr("""Pero ¿de dónde salen? En este vídeo vamos a verlas, no
                            """But where do they come from? In this video we are goin
    self.pausa(tracker.duration/2)
    self.play(FadeOut(ocho), run_time=tracker.duration/4)
    self.play(Create(recta), run_time=tracker.duration/4)

with self.voiceover(text=tr("""Para ello, un número real será un punto de una recta e
                            """For that, a real number will be a point on a line on w
    self.play(Indicate(recta.numbers[0], color=COLOR_RESULTADO), Indicate(recta.numbe
              run_time=tracker.duration/3)
    self.play(Create(arco), run_time=tracker.duration/3)
    self.play(FadeIn(punto_a), FadeIn(etq_a), FadeOut(arco), run_time=tracker.duratio

with self.voiceover(text=tr("""Veremos que las sumas y los productos de números reale
                            """We will see that sums and products of real numbers can
    reglas = Tex(tr("regla y compás $\\;\\longrightarrow\\;$ reglas para manipular sí
                    "ruler and compass $\\;\\longrightarrow\\;$ rules for manipulatin
                 font_size=40).set_color(COLOR_ESCALAR).next_to(recta, DOWN, buff=0.9
    self.pausa(tracker.duration/2)
    self.play(Write(reglas), run_time=tracker.duration/2)

self.pausa()
```

1.5.3. Escena 2 - La suma, trasladando segmentos

Sobre una recta con el cero y el uno. Un número es un segmento con el origen en el cero y el otro extremo en el número. Sumar es poner (con el compás) el origen del segundo segmento en el extremo del primero: el extremo del segmento trasladado es la suma, es decir, un nuevo segmento con origen en el cero. Conmutativa, asociativa, cero y opuesto (el segmento abatido al otro lado del cero); con la misma regla de la suma, 'a' más su opuesto acaba en el cero, así que no hace falta definir la resta. Durante cada traslación, `compas()` marca con un punto el origen del segmento que

se mueve.

```
self.creditos()
cabecera = Tex(tr("La suma, con regla y compás", "Addition, with ruler and compass"),
               font_size=44).set_color(COLOR_TITULO).to_edge(UP)
self.add(cabecera)

a, b, c = 2, 3, 1.5
recta = NumberLine(x_range=[-4, 9, 1], length=12.5, include_numbers=False).shift(1.9*
recta.add_numbers([0, 1], font_size=36)
u = recta.n2p(1)[0] - recta.n2p(0)[0]

def altura(fila):
    return 1.2 + 0.55*fila

def segmento(desde, hasta, color, fila=0):
    return Line(recta.n2p(desde) + altura(fila)*UP, recta.n2p(hasta) + altura(fila)*U
                color=color, stroke_width=10)

def rotulo(seg, texto, color):
    return MathTex(texto, color=color, font_size=42).next_to(seg, UP, buff=0.12)

def compas(seg_origen, valor, desde, fila=0, run_time=1.5, antes=None, abatir=None):
    """Copia el segmento `seg_origen` (de longitud |valor|; su origen es el extremo
    `start`), marca el origen con un punto, levanta la copia en vertical sobre el
    origen, la traslada hasta poner el origen en el punto `desde` y la abate con el
    compás: en el sentido de las agujas del reloj si valor>0, en el contrario si
    valor<0 (el segmento queda orientado como estaba).
    Así se ve desde el principio que el segmento trasladado mide lo mismo y cuál de
    sus extremos es el origen. Devuelve la copia y un grupo con el arco y el punto
    del origen (para borrarlos después).
    Para que el movimiento acompañe a la voz, `antes` y `abatir` son marcas
    (<bookmark mark='x'/>) a las que se espera antes de levantar el segmento y antes
    de abatirlo; deben usarse dentro de un `with self.voiceover(...)`."""
    base = recta.n2p(desde) + altura(fila)*UP
    copia = seg_origen.copy()
    inicio = copia.get_start()
    origen = Dot(inicio, color=copia.get_color(), radius=0.11)
    movil = VGroup(copia, origen)
    hacia_derecha = copia.get_end()[0] > inicio[0]
```

```
    if antes is not None:
        self.wait_until_bookmark(antes)
    self.play(FadeIn(origen, scale=2), run_time=run_time/8)
    self.play(Rotate(movil, angle=PI/2 if hacia_derecha else -PI/2, about_point=inici
    self.play(movil.animate.shift(base - inicio), run_time=run_time/4)
    if abatir is not None:
        self.wait_until_bookmark(abatir)
    angulo = -PI/2 if valor > 0 else PI/2
    arco = Arc(radius=abs(valor)*u, start_angle=PI/2, angle=angulo, arc_center=base,
               color=GRAY_B, stroke_width=1.5)
    self.play(Rotate(movil, angle=angulo, about_point=base), Create(arco), run_time=3
    return copia, VGroup(arco, origen)

def marca(valor, texto, color=COLOR_RESULTADO):
    punto = Dot(recta.n2p(valor), color=color, radius=0.09)
    etq = MathTex(texto, color=color, font_size=42).next_to(punto, DOWN, buff=0.45)
    return VGroup(punto, etq)

def regla(texto):
    return MathTex(texto, font_size=52, color=COLOR_REGLA).next_to(cabecera, DOWN, bu

seg_a = segmento(0, a, COLOR_A)
etq_a = rotulo(seg_a, "a", COLOR_A)
origen_a = Dot(seg_a.get_start(), color=COLOR_A, radius=0.11)

with self.voiceover(text=tr("""Empecemos por la suma. Fijados el cero y el uno, tomem
                               """Let us start with addition. Once zero and one are fixe
    self.play(Create(recta), run_time=1.5)
    self.play(FadeIn(marca(a, "a", COLOR_A)[0]), run_time=0.6)
    self.wait_until_bookmark("origen")
    self.play(FadeIn(origen_a, scale=2), Indicate(recta.numbers[0], color=COLOR_RESUL
    self.wait_until_bookmark("extremo")
    self.play(Create(seg_a), FadeIn(etq_a), run_time=1.2)
    self.play(FadeOut(origen_a), run_time=0.4)

seg_b = segmento(0, b, COLOR_B, fila=1)
etq_b = rotulo(seg_b, "b", COLOR_B)
regla1 = regla("a+b=b+a")

with self.voiceover(text=tr("""Y aquí tenemos otro número, b: otro segmento con el or
```

```
                            """And here we have another number, b: another segment wi
    self.play(Create(seg_b), FadeIn(etq_b), run_time=tracker.duration)

with self.voiceover(text=tr("""Para sumar 'a' y b, <bookmark mark='traslada'/>traslad
                            """To add the two numbers, <bookmark mark='traslada'/>we
    self.anuncia_regla(regla1, run_time=0.8)
    seg_b_tras, arco1 = compas(seg_b, b, a, fila=0, run_time=2.4, antes="traslada", a
    etq_b_tras = rotulo(seg_b_tras, "b", COLOR_B)
    self.add(etq_b_tras)

# la suma es un nuevo segmento con origen en el cero (en blanco, debajo de los dos su
marca_ab = marca(a + b, "a+b")
seg_ab = segmento(0, a + b, COLOR_RESULTADO, fila=-1).set_stroke(width=6)
with self.voiceover(text=tr("""El extremo del segmento trasladado es la suma, 'a' más
                            """The end of the moved segment is the sum, 'a' plus 'b'.
    self.play(FadeIn(marca_ab), run_time=0.8)
    self.wait_until_bookmark("nuevo")
    self.play(Create(seg_ab), run_time=1.5)

with self.voiceover(text=tr("""Hagámoslo ahora en el otro orden: partimos de b <bookm
                            """Let us now do it in the other order: we start from 'b'
    seg_a_tras, arco2 = compas(seg_a, a, b, fila=1, run_time=2.4, antes="traslada")
    etq_a_tras = rotulo(seg_a_tras, "a", COLOR_A)
    self.add(etq_a_tras)

linea = DashedLine(recta.n2p(a + b) + 0.1*UP, recta.n2p(a + b) + altura(1.6)*UP, colo
with self.voiceover(text=tr("""Llegamos exactamente al mismo punto. Es la propiedad c
                            """We arrive at exactly the same point. This is the commu
    self.play(Create(linea), FadeOut(arco1, arco2), run_time=tracker.duration/3)
    self.destaca_regla(regla1)

self.play(FadeOut(seg_b, etq_b, seg_b_tras, etq_b_tras, seg_a_tras, etq_a_tras, linea

# asociativa: las dos agrupaciones se construyen en el orden en que se enuncian.
# (a+b)+c en la fila 0; a+(b+c) en la fila 1, con b+c construido antes en la fila 2.
seg_b2 = segmento(0, b, COLOR_B, fila=2)
seg_c3 = segmento(0, c, COLOR_C, fila=3)
etq_b2, etq_c3 = rotulo(seg_b2, "b", COLOR_B), rotulo(seg_c3, "c", COLOR_C)
regla2 = regla("a+(b+c)=(a+b)+c")
self.anuncia_regla(regla2, regla1)
```

```
self.play(FadeIn(seg_b2, etq_b2, seg_c3, etq_c3))

with self.voiceover(text=tr("""Con tres números podemos agrupar de dos maneras. <book
                            """With three numbers we can group in two ways. <bookmark
    self.wait_until_bookmark("ab")
    seg_b0, arco1 = compas(seg_b2, b, a, fila=0, run_time=1.6)
    etq_b0 = rotulo(seg_b0, "b", COLOR_B)
    self.add(etq_b0)
    llave1 = Brace(VGroup(seg_a, seg_b0), DOWN, buff=0.1, color=GRAY_B)
    etq_llave1 = llave1.get_tex("a+b").set_color(GRAY_B)
    self.play(GrowFromCenter(llave1), FadeIn(etq_llave1), FadeOut(arco1), run_time=0.
    self.wait_until_bookmark("c")
    seg_c0, arco2 = compas(seg_c3, c, a + b, fila=0, run_time=1.6)
    etq_c0 = rotulo(seg_c0, "c", COLOR_C)
    self.add(etq_c0)
    self.play(FadeOut(arco2), run_time=0.4)

seg_a1 = segmento(0, a, COLOR_A, fila=1)
etq_a1 = rotulo(seg_a1, "a", COLOR_A)
with self.voiceover(text=tr("""O primero b más c, poniendo el origen de c en el extre
                            """Or first b plus c, placing the origin of c at the end
    seg_c2, arco3 = compas(seg_c3, c, b, fila=2, run_time=1.6)
    etq_c2 = rotulo(seg_c2, "c", COLOR_C)
    self.add(etq_c2)
    grupo_bc = VGroup(seg_b2, etq_b2, seg_c2, etq_c2)
    llave2 = Brace(grupo_bc, UP, buff=0.1, color=GRAY_B)
    etq_llave2 = llave2.get_tex("b+c").set_color(GRAY_B)
    self.play(FadeOut(arco3, seg_c3, etq_c3), run_time=0.4)
    self.play(GrowFromCenter(llave2), FadeIn(etq_llave2), run_time=0.8)
    grupo_bc.add(llave2, etq_llave2)
    self.wait_until_bookmark("a")
    self.play(FadeIn(seg_a1, etq_a1), run_time=0.6)
    # el segmento b+c, ya construido, se traslada en bloque hasta el extremo de a
    self.play(grupo_bc.animate.shift(recta.n2p(a) - recta.n2p(0) + (altura(1) - altur

marca_abc = marca(a + b + c, "a+b+c")
with self.voiceover(text=tr("""El punto final es el mismo. Es la propiedad asociativa
                            """The end point is the same. This is the associative pro
    self.play(FadeIn(marca_abc), run_time=tracker.duration/3)
    self.destaca_regla(regla2)
```

```
self.play(FadeOut(seg_b0, etq_b0, seg_c0, etq_c0, seg_a1, etq_a1, grupo_bc, llave1, e

# cero: un segmento de longitud cero (un punto) con el origen en el cero, que se tras
cero = Dot(recta.n2p(0) + altura(1)*UP, color=COLOR_B, radius=0.12)
regla3 = regla("a+0=a")
with self.voiceover(text=tr("""Sumar cero es sumar un segmento de longitud cero: <boo
                            """Adding zero means adding a segment of length zero: <bo
    self.anuncia_regla(regla3, regla2, run_time=0.8)
    self.wait_until_bookmark("punto")
    self.play(FadeIn(cero, scale=3), run_time=0.8)
    self.wait_until_bookmark("traslada")
    self.play(cero.animate.move_to(recta.n2p(a) + altura(0)*UP), run_time=1.5)
    self.wait_until_bookmark("quietos")
    self.play(Flash(cero, color=COLOR_RESULTADO), run_time=0.8)
    self.wait_until_bookmark("neutro")
    self.destaca_regla(regla3)
    self.play(FadeOut(cero), run_time=0.5)

self.pausa(1.2)   # respiro antes de cambiar de propiedad (1,5 s resultaba largo)

# opuesto: el mismo segmento abatido al lado contrario del cero (origen en el cero; s
regla4 = regla("a+(-a)=0")
with self.voiceover(text=tr("""Todo número tiene un opuesto. Para construir el opuest
                            """Every number has an opposite. To construct the opposit
    self.anuncia_regla(regla4, regla3, run_time=0.8)
    seg_ma, arco1 = compas(seg_a, -a, 0, fila=0, run_time=2.4, antes="compas", abatir
    seg_ma.set_opacity(0.6)
    self.wait_until_bookmark("obtenemos")
    marca_ma = marca(-a, "-a", COLOR_A)
    self.play(FadeIn(marca_ma), FadeOut(arco1[0]), run_time=0.8)   # el punto del ori

with self.voiceover(text=tr("""Sumemos ahora 'a' y su opuesto con la regla de siempre
                            """Now let us add 'a' and its opposite with the usual rul
    seg_ma1, arco2 = compas(seg_ma, -a, a, fila=1, run_time=2.4, antes="traslada", ab
    punto_0 = Dot(recta.n2p(0), color=COLOR_RESULTADO, radius=0.09)
    linea0 = DashedLine(recta.n2p(0) + 0.1*UP, recta.n2p(0) + altura(1.6)*UP, color=C
    self.play(Create(linea0), FadeIn(punto_0), Indicate(recta.numbers[0], color=COLOR
    self.wait_until_bookmark("regla")
    self.destaca_regla(regla4)
```

```
with self.voiceover(text=tr("""Acabamos de ver cuatro propiedades de la suma. Y las c
                            """We have just seen four properties of addition. And all
    self.pausa(tracker.duration)

self.pausa()
```

1.5.4. Escena 3 - El producto: la figura del vídeo 4 es una homotecia

Recordamos la construcción de Descartes (vídeo 4): recta por (a,0) y (0,1); la paralela por (0,b) corta el eje horizontal en ab. Esa figura convierte cada número del eje vertical en su producto por a. El uno es neutro; el inverso de a es el número del eje vertical cuya paralela corta al eje horizontal en el uno (no se usa en el vídeo 6, pero es una propiedad más de los reales); la distributiva se ve porque la construcción respeta la concatenación de segmentos.

```
self.creditos()
cabecera = Tex(tr("El producto, con regla y compás", "Multiplication, with ruler and
               font_size=44).set_color(COLOR_TITULO).to_edge(UP)
self.add(cabecera)

a, b, c = 2, 1.5, 2
ax = Axes(x_range=[-0.5, 8.5, 1], y_range=[-0.5, 4.2, 1], x_length=10, y_length=10*4.
          tips=False, axis_config={"include_ticks": True})
ax.shift(np.array([-4.8, -2.45, 0]) - ax.c2p(0, 0))   # origen abajo a la izquierda
O = ax.c2p(0, 0)
ux = ax.c2p(1, 0)[0] - O[0]

def recta_por(p, q, color=GRAY_B, ext=1.25):
    return Line(ax.c2p(*p), ax.c2p(*q), color=color, stroke_width=2.5).scale(ext)

def marca_x(valor, texto, color=COLOR_RESULTADO, buff=0.35):
    punto = Dot(ax.c2p(valor, 0), color=color, radius=0.09)
    etq = MathTex(texto, color=color, font_size=42).next_to(punto, DOWN, buff=buff)
    return VGroup(punto, etq)

def marca_y(valor, texto, color=COLOR_RESULTADO):
    punto = Dot(ax.c2p(0, valor), color=color, radius=0.09)
```

```
    etq = MathTex(texto, color=color, font_size=42).next_to(punto, LEFT, buff=0.3)
    return VGroup(punto, etq)

def seg_x(desde, hasta, color, altura=0.0):
    return Line(ax.c2p(desde, 0) + altura*UP, ax.c2p(hasta, 0) + altura*UP, color=col

def seg_y(desde, hasta, color):
    return Line(ax.c2p(0, desde), ax.c2p(0, hasta), color=color, stroke_width=10)

def regla(texto):
    return MathTex(texto, font_size=52, color=COLOR_REGLA).next_to(cabecera, DOWN, bu

with self.voiceover(text=tr("""Pasemos al producto. En el vídeo anterior vimos una co
                            """Let us move on to multiplication. In the previous vide
    self.play(Create(ax), run_time=tracker.duration/2)

m_a, m_b, m_1 = marca_x(a, "a", COLOR_A), marca_y(b, "b", COLOR_B), marca_y(1, "1")
with self.voiceover(text=tr("""Ponemos 'a' en el eje horizontal y b en el eje vertica
                            """We place a on the horizontal axis and 'b' on the verti
    self.play(FadeIn(m_a), run_time=tracker.duration/2)
    self.play(FadeIn(m_b), run_time=tracker.duration/2)

diagonal = recta_por((a, 0), (0, 1))
with self.voiceover(text=tr("""Trazamos la recta que une el punto 'a' del eje horizon
                            """We draw the line joining point a on the horizontal axi
    self.play(FadeIn(m_1), run_time=tracker.duration/4)
    self.play(Create(diagonal), run_time=tracker.duration/2)

paralela_b = recta_por((a*b, 0), (0, b), color=COLOR_B)
with self.voiceover(text=tr("""Y trazamos la paralela que pasa por b. <break time="60
                            """And we draw the parallel line through 'b'. <break time
    self.play(TransformFromCopy(diagonal, paralela_b), run_time=tracker.duration*0.6)

m_ab = marca_x(a*b, "ab")
with self.voiceover(text=tr("""Donde esa paralela corta al eje horizontal está el pro
                            """Where that parallel meets the horizontal axis is the p
    self.play(FadeIn(m_ab, scale=2), run_time=tracker.duration/2)

# la construcción como homotecia
t = ValueTracker(b)
```

```
paralela_t = always_redraw(lambda: recta_por((a*t.get_value(), 0), (0, t.get_value())
punto_t = always_redraw(lambda: Dot(ax.c2p(a*t.get_value(), 0), color=COLOR_ESCALAR,
punto_ty = always_redraw(lambda: Dot(ax.c2p(0, t.get_value()), color=COLOR_ESCALAR, r
with self.voiceover(text=tr("""Fíjese en lo que hace esta figura: a cada número del e
                            """Notice what this figure does: to each number on the ve
    self.add(paralela_t, punto_t, punto_ty)
    self.play(t.animate.set_value(2.8), run_time=tracker.duration*0.4, rate_func=ther
    self.play(t.animate.set_value(0.4), run_time=tracker.duration*0.4, rate_func=ther

regla1 = regla("1a=a")
with self.voiceover(text=tr("""Si tomamos el uno, la paralela es la propia recta inic
                            """If we take one, the parallel line is the initial line
    self.anuncia_regla(regla1, run_time=0.8)
    self.play(t.animate.set_value(1), run_time=tracker.duration/3)
    self.play(Indicate(m_a, color=COLOR_RESULTADO), run_time=tracker.duration/4)
    self.destaca_regla(regla1)

# inverso: la paralela que corta al eje horizontal en el uno
regla_inv = regla(r"a\,a^{-1}=1")
m_1x = marca_x(1, "1")
m_inv = marca_y(1/a, "a^{-1}", COLOR_A)
with self.voiceover(text=tr("""Hay otra propiedad del producto: todo número distinto
                            """There is another property of multiplication: every non
    self.anuncia_regla(regla_inv, regla1, run_time=0.8)
    self.wait_until_bookmark("mueve")
    self.play(FadeIn(m_1x, scale=2), run_time=0.6)
    self.play(t.animate.set_value(1/a), run_time=2)
    self.wait_until_bookmark("inverso")
    self.play(FadeIn(m_inv, scale=2), run_time=0.8)
    self.destaca_regla(regla_inv)
    self.remove(paralela_t, punto_t, punto_ty)
    self.play(FadeOut(m_1x, m_inv), run_time=0.5)

self.pausa(1.2)   # respiro antes de cambiar de propiedad

# distributiva
regla2 = regla("a(b+c)=ab+ac")
seg_b_y = seg_y(0, b, COLOR_B)
seg_c_y = seg_y(b, b + c, COLOR_C)
m_bc = marca_y(b + c, "b+c", COLOR_C)
```

```
with self.voiceover(text=tr("""Ahora sumemos otro número c a b, trasladándolo con el
                            """Now let us add a third number, c, to b, moving it with
    self.anuncia_regla(regla2, regla_inv, run_time=0.8)
    self.play(Create(seg_b_y), run_time=tracker.duration/4)
    self.play(Create(seg_c_y), FadeIn(m_bc), run_time=tracker.duration/2)

paralela_bc = recta_por((a*(b + c), 0), (0, b + c), color=COLOR_C)
m_abc = marca_x(a*(b + c), "a(b+c)")
with self.voiceover(text=tr("""La paralela que pasa por 'b más c' nos da el producto
                            """The parallel through b plus c gives us the product 'a'
    self.play(TransformFromCopy(diagonal, paralela_bc), run_time=tracker.duration/2)
    self.play(FadeIn(m_abc, scale=2), run_time=tracker.duration/4)

seg_ab_x = seg_x(0, a*b, COLOR_B, altura=0.18)
seg_ac_x = seg_x(a*b, a*(b + c), COLOR_C, altura=0.18)
with self.voiceover(text=tr("""En el eje horizontal, <bookmark mark='s1'/>entre cero
                            """On the horizontal axis, <bookmark mark='s1'/>between z
    self.wait_until_bookmark("s1")
    self.play(Create(seg_ab_x), run_time=1)
    self.play(Indicate(seg_ab_x, color=COLOR_RESULTADO, scale_factor=1.05), run_time=
    self.wait_until_bookmark("s2")
    self.play(Create(seg_ac_x), run_time=1)
    self.play(Indicate(seg_ac_x, color=COLOR_RESULTADO, scale_factor=1.05), run_time=

paralela_c = recta_por((a*c, 0), (0, c), color=COLOR_C).set_opacity(0.6)
m_c = marca_y(c, "c", COLOR_C)
m_ac = marca_x(a*c, "ac", COLOR_C, buff=0.8)
with self.voiceover(text=tr("""¿Cuánto mide ese segmento? <bookmark mark='aparte'/>Co
                            """How long is that segment? <bookmark mark='aparte'/>Let
    self.play(Indicate(seg_ac_x, color=COLOR_RESULTADO, scale_factor=1.05), run_time=
    self.wait_until_bookmark("aparte")
    self.play(FadeIn(m_c), run_time=tracker.duration/4)
    self.play(TransformFromCopy(diagonal, paralela_c), run_time=tracker.duration/3)
    self.play(FadeIn(m_ac, scale=2), run_time=tracker.duration/4)

# el compás, como en la escena 2: la copia (en blanco) se levanta sobre su extremo iz
# se lleva al origen y se abate hacia la derecha, en el sentido de las agujas del rel
copia = seg_ac_x.copy().set_color(COLOR_RESULTADO)
with self.voiceover(text=tr("""Y traslademos el segmento con el compás hasta el orige
                            """And let us move the segment with the compass to the or
```

```
        inicio = copia.get_start()
        origen = O + 0.18*UP
        self.play(Rotate(copia, angle=PI/2, about_point=inicio), run_time=0.7)
        self.play(copia.animate.shift(origen - inicio), run_time=0.7)
        arco = Arc(radius=a*c*ux, start_angle=PI/2, angle=-PI/2, arc_center=origen, color
        self.play(Rotate(copia, angle=-PI/2, about_point=origen), Create(arco), run_time=
        self.wait_until_bookmark("coincide")
        self.play(FadeOut(arco), Indicate(m_ac, color=COLOR_RESULTADO), run_time=1)

    # cada segmento vertical "viaja" hasta su imagen en el eje horizontal; después, las d
    with self.voiceover(text=tr("""No es casualidad: <bookmark mark='traslada'/>la figura
                                """This is no coincidence: <bookmark mark='traslada'/>the
        self.play(FadeOut(copia), run_time=0.5)
        self.wait_until_bookmark("traslada")
        viaje_b, viaje_c = seg_b_y.copy(), seg_c_y.copy()
        self.play(Transform(viaje_b, seg_ab_x), run_time=1.3)
        self.play(Transform(viaje_c, seg_ac_x), run_time=1.3)
        self.remove(viaje_b, viaje_c)
        # "una suma": los dos sumandos se apartan y se separan un instante, y vuelven a e
        self.wait_until_bookmark("sumav")
        cb, cc = seg_b_y.copy(), seg_c_y.copy()
        self.play(cb.animate.shift(0.9*LEFT), cc.animate.shift(0.9*LEFT + 0.5*UP), run_ti
        self.play(cb.animate.shift(0.9*RIGHT), cc.animate.shift(0.9*RIGHT + 0.5*DOWN), ru
        self.play(Indicate(m_bc, color=COLOR_RESULTADO), run_time=0.6)
        self.remove(cb, cc)
        self.wait_until_bookmark("sumah")
        cab, cac = seg_ab_x.copy(), seg_ac_x.copy()
        self.play(cab.animate.shift(0.8*UP + 0.4*LEFT), cac.animate.shift(0.8*UP + 0.4*RI
        self.play(cab.animate.shift(0.8*DOWN + 0.4*RIGHT), cac.animate.shift(0.8*DOWN + 0
        self.play(Indicate(m_abc, color=COLOR_RESULTADO), run_time=0.6)
        self.remove(cab, cac)
        self.wait_until_bookmark("distributiva")
        self.destaca_regla(regla2)

    self.pausa()
```

1.5.5. Escena 4 - Conmutativa y asociativa del producto: se puede, pero es engorroso

El momento pedagógico: las dos construcciones acaban en el mismo punto, pero hacen falta muchas rectas y arcos. Euclides razonaba con figuras; Descartes dio el salto a los símbolos.

```
self.creditos()
cabecera = Tex(tr("Dos propiedades incómodas", "Two uncomfortable properties"),
               font_size=44).set_color(COLOR_TITULO).to_edge(UP)
self.add(cabecera)

a, b, c = 2, 1.5, 2.5
ax = Axes(x_range=[-0.5, 8.5, 1], y_range=[-0.5, 4.2, 1], x_length=10, y_length=10*4.
          tips=False, axis_config={"include_ticks": True})
ax.shift(np.array([-4.8, -2.45, 0]) - ax.c2p(0, 0))   # origen abajo a la izquierda
O = ax.c2p(0, 0)
ux = ax.c2p(1, 0)[0] - O[0]

def recta_por(p, q, color=GRAY_B, ext=1.25):
    return Line(ax.c2p(*p), ax.c2p(*q), color=color, stroke_width=2.5).scale(ext)

def marca_x(valor, texto, color=COLOR_RESULTADO, buff=0.35):
    punto = Dot(ax.c2p(valor, 0), color=color, radius=0.09)
    etq = MathTex(texto, color=color, font_size=42).next_to(punto, DOWN, buff=buff)
    return VGroup(punto, etq)

def marca_y(valor, texto, color=COLOR_RESULTADO):
    punto = Dot(ax.c2p(0, valor), color=color, radius=0.09)
    etq = MathTex(texto, color=color, font_size=42).next_to(punto, LEFT, buff=0.3)
    return VGroup(punto, etq)

def arco_x_a_y(valor, color):
    """Arco de compás, centrado en el origen, que lleva el punto (valor,0) a (0,valor
    return Arc(radius=valor*ux, start_angle=0, angle=PI/2, arc_center=O, color=color,

def regla(texto):
    return MathTex(texto, font_size=52, color=COLOR_REGLA).next_to(cabecera, DOWN, bu

def multiplica(x, y, color, velocidad=1.0):
    """Construye x·y: recta por (x,0),(0,1) y paralela por (0,y). Devuelve los objeto
```

```
    d = recta_por((x, 0), (0, 1))
    p = recta_por((x*y, 0), (0, y), color=color)
    self.play(Create(d), run_time=0.8*velocidad)
    self.play(TransformFromCopy(d, p), run_time=0.8*velocidad)
    return VGroup(d, p)

self.play(Create(ax), run_time=1)
m_a, m_b, m_1 = marca_x(a, "a", COLOR_A), marca_y(b, "b", COLOR_B), marca_y(1, "1")
self.play(FadeIn(m_a, m_b, m_1), run_time=0.8)
regla1 = regla("ab=ba")
regla2 = regla("a(bc)=(ab)c")

with self.voiceover(text=tr("""¿Y la conmutativa? a por b igual a b por a. Nuestra co
                            """What about commutativity? 'a' times b equals b times a
    self.anuncia_regla(regla1, run_time=0.8)
    rectas_ab = multiplica(a, b, COLOR_B, velocidad=tracker.duration/4)
    m_ab = marca_x(a*b, "ab")
    self.play(FadeIn(m_ab, scale=2), run_time=tracker.duration/5)

with self.voiceover(text=tr("""Para calcular b por a necesitamos intercambiar los pap
                            """To compute b times 'a' we have to swap the roles: we c
    self.play(rectas_ab.animate.set_stroke(opacity=0.35), run_time=tracker.duration/6
    arco_b, arco_a = arco_x_a_y(b, COLOR_B), arco_x_a_y(a, COLOR_A)
    m_b_x, m_a_y = marca_x(b, "b", COLOR_B, buff=0.8), marca_y(a, "a", COLOR_A)
    self.play(Create(arco_b), FadeIn(m_b_x), run_time=tracker.duration/3)
    self.play(Create(arco_a), FadeIn(m_a_y), run_time=tracker.duration/3)

with self.voiceover(text=tr("""Y repetimos la construcción: la recta por b y por uno,
                            """And we repeat the construction: the line through 'b' a
    rectas_ba = multiplica(b, a, COLOR_A, velocidad=tracker.duration/3)

with self.voiceover(text=tr("""Son dos construcciones distintas, <bookmark mark='mism
                            """They are two different constructions, <bookmark mark='
    self.wait_until_bookmark("mismo")
    self.play(Indicate(m_ab, color=COLOR_RESULTADO, scale_factor=1.6), run_time=1)
    self.wait_until_bookmark("regla")
    self.destaca_regla(regla1)

self.play(FadeOut(rectas_ab, rectas_ba, arco_a, arco_b, m_b_x, m_a_y, m_ab, m_b), run
```

```
# asociativa
with self.voiceover(text=tr("""La asociativa es peor: a por, b por c, frente a, a por
                            """Associativity is worse: the number 'a' times the produ
    self.anuncia_regla(regla2, regla1, run_time=0.8)
    m_c, m_b_x = marca_y(c, "c", COLOR_C), marca_x(b, "b", COLOR_B, buff=0.8)
    self.play(FadeIn(m_c, m_b_x), run_time=tracker.duration/5)
    r1 = multiplica(b, c, COLOR_C, velocidad=tracker.duration/8)
    m_bc = marca_x(b*c, "bc", COLOR_C, buff=0.8)
    self.play(FadeIn(m_bc), run_time=tracker.duration/10)
    arco1 = arco_x_a_y(b*c, COLOR_C)
    m_bc_y = marca_y(b*c, "bc", COLOR_C)
    self.play(Create(arco1), FadeIn(m_bc_y), run_time=tracker.duration/5)
    r2 = multiplica(a, b*c, COLOR_C, velocidad=tracker.duration/8)
    m_abc = marca_x(a*b*c, "a(bc)", COLOR_C)
    self.play(FadeIn(m_abc, scale=2), run_time=tracker.duration/10)

with self.voiceover(text=tr("""Y por el otro camino: a por b, lo llevamos al eje vert
                            """And the other way round: we compute the product 'a' ti
    self.play(VGroup(r1, r2, arco1).animate.set_stroke(opacity=0.3), run_time=tracker
    r3 = multiplica(a, b, COLOR_B, velocidad=tracker.duration/8)
    m_ab = marca_x(a*b, "ab", COLOR_B)
    self.play(FadeIn(m_ab), run_time=tracker.duration/10)
    arco2 = arco_x_a_y(a*b, COLOR_B)
    m_ab_y = marca_y(a*b, "ab", COLOR_B)
    self.play(Create(arco2), FadeIn(m_ab_y), run_time=tracker.duration/5)
    m_c_x = marca_x(c, "c", COLOR_C, buff=0.8)
    self.play(FadeIn(m_c_x), run_time=tracker.duration/10)
    r4 = multiplica(c, a*b, COLOR_B, velocidad=tracker.duration/8)
    m_abc2 = MathTex("(ab)c", color=COLOR_B, font_size=42).next_to(m_abc, DOWN, buff=
    self.play(FadeIn(m_abc2), run_time=tracker.duration/10)

with self.voiceover(text=tr("""Mismo punto. Pero cuente: <bookmark mark='rectas'/>cua
                            """Same point. But count: <bookmark mark='rectas'/>four l
    self.play(Indicate(m_abc, color=COLOR_RESULTADO, scale_factor=1.5), run_time=1)
    # cada cosa que cuenta la voz parpadea cuando la nombra; la fórmula, al final
    self.wait_until_bookmark("rectas")
    self.play(*[Indicate(r[0], color=COLOR_RESULTADO, scale_factor=1.03) for r in (r1
    self.wait_until_bookmark("paralelas")
    self.play(*[Indicate(r[1], color=COLOR_RESULTADO, scale_factor=1.03) for r in (r1
    self.wait_until_bookmark("arcos")
```

```
    self.play(Indicate(arco1, color=COLOR_RESULTADO, scale_factor=1.03), Indicate(arc
    self.wait_until_bookmark("tres")
    self.destaca_regla(regla2)

self.play(*[FadeOut(mob) for mob in self.mobjects if mob is not cabecera], run_time=0

euclides = Tex(tr("Euclides, siglo III a.\\,C.: álgebra con figuras", "Euclid, 3rd ce
               font_size=42)
descartes = Tex(tr("Descartes, 1637: los segmentos son números, y se opera con símbol
                   "Descartes, 1637: segments are numbers, and one operates with symb
                font_size=42).set_color(COLOR_ESCALAR)
escuela = Tex(tr("La escuela: las reglas se comprueban con ejemplos numéricos, y se u
                 "School: the rules are checked on numerical examples, and used"),
              font_size=42)
historia = VGroup(euclides, descartes, escuela).arrange(DOWN, buff=0.6).move_to(0.3*D
for h in historia:
    h.scale_to_fit_width(min(h.width, config.frame_width*0.9))

with self.voiceover(text=tr("""Todas estas propiedades se pueden ver con regla y comp
                            """All these properties can be seen with ruler and compas
    self.play(Write(euclides), run_time=tracker.duration/2)

with self.voiceover(text=tr("""Hubo que esperar casi dos mil años a que Descartes die
                            """It took almost two thousand years for Descartes to mak
    self.play(Write(descartes), run_time=tracker.duration/2)

with self.voiceover(text=tr("""Esa es la tradición que usted recibió en la escuela: a
                            """That is the tradition you received at school: there th
    self.play(Write(escuela), run_time=tracker.duration/2)

self.pausa()
```

1.5.6. Escena 5 - Las nueve reglas y «la notación lo es todo»

Lista de las nueve propiedades (las ocho primeras son las que abren el vídeo 6), cita de Van Loan del prefacio del libro y anuncio del siguiente vídeo (sin pregunta en la voz: Azure la entona mal; la pregunta queda en pantalla).

```
self.creditos()
```

```
cabecera = Tex(tr("Las reglas del juego", "The rules of the game"),
               font_size=44).set_color(COLOR_TITULO).to_edge(UP)
self.add(cabecera)

sean = Tex(tr(r"Para cualesquiera números reales $a$, $b$ y $c$:", r"For any real num
           tex_template=myTemplate, font_size=40).next_to(cabecera, DOWN, buff=0.4)

propiedades = [r"a+b=b+a",
               r"a+(b+c)=(a+b)+c",
               r"a+0=a",
               r"a+(-a)=0",
               r"ab=ba",
               r"a(b+c)=ab+ac",
               r"a(bc)=(ab)c",
               r"1a=a",
               r"a\,a^{-1}=1\;\;(a\neq 0)"]

lista = VGroup(*[MathTex(f"{k+1}.\;\;", p, tex_template=myTemplate, font_size=44)
                 for k, p in enumerate(propiedades)])

# dos columnas (suma: 4 filas; producto: 5 filas) sobre una misma rejilla de filas
columna_suma     = VGroup(*lista[:4]).arrange(DOWN, aligned_edge=LEFT, buff=0.35)
columna_producto = VGroup(*lista[4:]).arrange(DOWN, aligned_edge=LEFT, buff=0.35)
cuadro = VGroup(columna_suma, columna_producto).arrange(RIGHT, buff=1.4, aligned_edge
cuadro.next_to(sean, DOWN, buff=0.5).set_x(0)
paso = 0.68
y0 = lista[0].get_y()
for k in range(4):
    lista[k].set_y(y0 - k*paso)
for k in range(5):
    lista[4+k].set_y(y0 - k*paso)

with self.voiceover(text=tr("""Resumamos. Las dos operaciones con números reales se p
                            """Let us sum up. The two operations with real numbers ca
    self.play(FadeIn(sean), run_time=tracker.duration/2)

with self.voiceover(text=tr("""De la suma: es conmutativa, es asociativa, el cero es
                            """For addition: it is commutative, it is associative, ze
    for k in range(4):
        self.play(FadeIn(lista[k]), run_time=tracker.duration/4)
```

```
with self.voiceover(text=tr("""Del producto: es conmutativo, es distributivo respecto
                            """For multiplication: it is commutative, it is distribut
    for k in range(4, 9):
        self.play(FadeIn(lista[k]), run_time=tracker.duration/5)

cita = VGroup(Tex(r"``Notation is everything''", font_size=46).set_color(COLOR_ESCALA
              Tex("Charles F. Van Loan", font_size=34).set_color(GRAY_B)).arrange(DOW
cita.next_to(cuadro, DOWN, buff=0.5)

with self.voiceover(text=tr("""A partir de ahora no volveremos a la regla y el compás
                            """From now on we will not go back to ruler and compass.
    self.pausa(tracker.duration/2)
    self.play(FadeIn(cita), run_time=tracker.duration/2)

self.pausa(1.2)   # respiro tras la cita

pregunta = Tex(tr(r"¿Cumplen estas propiedades las operaciones con \emph{vectores}?",
                  r"Do the operations with \emph{vectors} satisfy these properties?")
               tex_template=myTemplate, font_size=44).set_color(COLOR_ESCALAR).next_t

with self.voiceover(text=tr("""En el próximo vídeo plantearemos la misma cuestión par
                            """In the next video we will raise the same question for
    self.play(FadeOut(cita), run_time=1)
    self.play(Write(pregunta), run_time=2)

self.pausa_media()
```

2. Trozos comunes de código

2.1. Carga de las librerías Manim, $\text{manim}_{\text{voiceover}}$ y NacAL

```
import os
import re
from manim import *
from contextlib import contextmanager
from manim_voiceover import VoiceoverScene
from manim_voiceover.helper import remove_bookmarks
from manim_voiceover.services.azure import AzureService
from manim_voiceover.services.gtts import GTTSService
```

```
import nacal as nc
import sympy as sp
```

2.2. Idioma del vídeo

El único chunk que difiere entre los dos ficheros generados.

```
LANG = "es"
```

```
LANG = "en"
```

2.3. Utilidades bilingües

- `tr(es, en)` devuelve la cadena del idioma del vídeo.

`VOCES_AZURE` voz de Azure de cada idioma. Español: Tristan HD (`es-ES-Tristan:DragonHDLatestNeural`) e inglés: Ollie HD (`en-GB-Ollie:DragonHDLatestNeural`), elegidas el 2026-10-12 tras comparar ocho voces de cada idioma (antes, Álvaro y Ryan). Las voces HD entienden mejor el contexto y cambian de ritmo; ya pausan solas en los dos puntos, así que `BREAK_DOS_PUNTOS` queda vacío para ellas.

`SERVICIO_VOZ` `azure` (de pago, calidad final) o `gtts` (gratis, para pruebas). Se cambia con la variable de entorno `VOZ`.

```
def tr(es, en):
    """Cadena en el idioma del vídeo."""
    return es if LANG == "es" else en
```

```
VOCES_AZURE  = {"es": "es-ES-Tristan:DragonHDLatestNeural", "en": "en-
GB-Ollie:DragonHDLatestNeural"}   # voces HD desde 2026-10-12
# Las voces "DragonHD" ya hacen una pausa larga en los dos puntos (medido: 0,7 s); a
BREAK_DOS_PUNTOS = "" if "DragonHD" in VOCES_AZURE[LANG] else '<break time="500ms"/>
SERVICIO_VOZ = os.environ.get("VOZ", "azure")
```

2.4. Código de colores de la serie

Los mismos colores en todos los vídeos, para que el espectador lea la pantalla sin esfuerzo (documentados en `NotacionNacal.org`).

```
COLOR_A         = BLUE      # primer objeto (número a, vector a)
COLOR_B         = ORANGE    # segundo objeto (contrasta con el azul)
```

```
COLOR_C          = GREEN     # tercer objeto
COLOR_ESCALAR    = YELLOW    # escalares, preguntas al espectador, tarjeta de pausa
COLOR_RESULTADO = WHITE     # resultado de una operación
COLOR_REGLA      = TEAL      # pasos que usan una propiedad de los números reales
COLOR_TITULO     = BLUE      # títulos y cabeceras
```

2.5. Uso del paquete de LaTeX con la notación del curso (`nacal.sty`)

Descargable desde `https://github.com/mbujosab/nacal-latex-package`. Todo `Tex`, `MathTex` o `Title` con macros de `nacal` necesita `tex_template=myTemplate`. Se define una sola vez, a nivel de módulo (los bloques de ensamblado lo colocan tras las utilidades bilingües).

```
myTemplate = TexTemplate()
myTemplate.add_to_preamble(r"""\usepackage{nacal} """)
```

Definición de mi escena con voz

Subclase de `VoiceoverScene` con pausas de distintas duraciones, el selector de voz, la marca de copyright y un `voiceover` que convierte los dos puntos de la narración en una pausa (Azure no la hace) y que quita del subtítulo las marcas, los `<break>` y las comillas simples de entonación (`'a'`) en español, y hace funcionar las marcas `<bookmark mark`'x'/>= también con gTTS (con Azure la voz da el instante exacto de cada palabra; con gTTS se estima repartiendo la duración). Para sincronizar una animación con una palabra concreta: marca en el texto y `self.wait_until_bookmark("x")` dentro del `with`. Su método `setup()` (que Manim ejecuta antes de `construct()`) llama a `self.voz()`, así que las clases de las escenas no tienen que hacerlo.

```
class MiEscenaConVoz(VoiceoverScene):
    def setup(self):
        """Manim lo ejecuta antes de construct(): fija la voz del idioma."""
        self.voz()

    def pausa_muy_corta(self, n=0.3):
        self.wait(n)
    def pausa_corta(self, n=0.5):
        self.wait(n)
    def pausa(self, n=1):
        self.wait(n)
```

```
def pausa_media(self, n=1.5):
    self.wait(n)
def pausa_larga(self, n=3):
    self.wait(n)
def pausa_muy_larga(self, n=5):
    self.wait(n)

@contextmanager
def voiceover(self, text=None, **kwargs):
    """Como VoiceoverScene.voiceover, con tres añadidos: los dos puntos producen
    una pausa en la voz (Azure no la hace por sí sola); el subtítulo conserva el
    texto original (sin las marcas <bookmark mark='x'/>); y con gTTS, que no da
    los tiempos de cada palabra, las marcas se estiman repartiendo la duración
    proporcionalmente al texto, de modo que self.wait_until_bookmark("x")
    funciona con los dos servicios (con Azure, con precisión)."""
    if text is not None:
        # el subtítulo no lleva ni marcas, ni pausas SSML escritas a mano (<break
        # ni las comillas simples que guían la entonación de Azure ('a', 'b más c
        # ingleses (Euclid's) no se tocan porque van pegados a una letra
        subtitulo = re.sub(r"\s*<break[^>]*/>\s*", " ", remove_bookmarks(text))
        subtitulo = re.sub(r"(?<!\w)'([^']{1,15})'(?!\w)", r"\1", subtitulo)
        kwargs.setdefault("subcaption", subtitulo)
        if SERVICIO_VOZ == "gtts":
            text = re.sub(r"\s*<break[^>]*/>\s*", ", ", text).replace(":", ".")
        elif BREAK_DOS_PUNTOS:
            text = text.replace(": ", ": " + BREAK_DOS_PUNTOS)
    with super().voiceover(text=text, **kwargs) as tracker:
        if text is not None and "<bookmark" in text and not getattr(tracker, "boo
            # gTTS guarda en la caché el texto sin marcas: se reponen y se estima
            tracker.data["input_text"] = text
            tracker.data["word_boundaries"] = tracker._get_fallback_word_boundari
            tracker._process_bookmarks()
        yield tracker

def anuncia_regla(self, nueva, anterior=None, run_time=1):
    """Muestra, atenuada, la fórmula que se va a ilustrar (en lugar de la anterio
    Queda a la vista durante la construcción; al terminar, destaca_regla() la enc
    nueva.set_opacity(0.45)
    if anterior is None:
        self.play(FadeIn(nueva), run_time=run_time)
```

```
            else:
                self.play(ReplacementTransform(anterior, nueva), run_time=run_time)
            return nueva

        def destaca_regla(self, regla, run_time=1.5):
            """La ilustración ha terminado: la fórmula pasa a opacidad plena y parpadea."
            self.play(regla.animate.set_opacity(1), run_time=run_time/3)
            self.play(Indicate(regla, color=COLOR_RESULTADO, scale_factor=1.15), run_time

        def voz(self, servicio=SERVICIO_VOZ):
            """Fija el servicio de voz según el idioma del vídeo."""
            if servicio == "gtts":
                self.set_speech_service(GTTSService(lang=LANG, tld="com"))
            else:
                self.set_speech_service(AzureService(voice=VOCES_AZURE[LANG]))

        <<Créditos en distintas partes de la pantalla>>

        <<Pausa para pensar>>

    def creditos(self, variante=1):
        def analisis_opcion_elegida(tipo):
            'Análisis de las opciones de eliminación elegidas'
            lista = [100,20,10,4,2,1]
            opcion = set()
            for t in lista:
                if (tipo - (tipo % t)) in lista:
                    opcion.add(tipo - (tipo % t))
                    tipo = tipo % t
            return opcion

        copyright = Tex(r"\textcopyright{\;} 2026\; Marcos Bujosa  ")

        if 1 in analisis_opcion_elegida(variante):
            stampDcha  = VGroup(copyright.copy()).rotate( PI/2).scale(0.5).to_edge(RIGHT,
            self.add(stampDcha)
        if 2 in analisis_opcion_elegida(variante):
            stampIzda  = VGroup(copyright.copy()).rotate(-PI/2).scale(0.5).to_edge(LEFT,
            self.add(stampIzda)
        if 4 in analisis_opcion_elegida(variante):
```

```
        stampBottom= VGroup(copyright.copy()).rotate(    0).scale(0.5).to_edge(DOWN,
        self.add(stampBottom)
    if 10 in analisis_opcion_elegida(variante):
        stampTop   = VGroup(copyright.copy()).rotate(    0).scale(0.5).to_edge(  UP,
        self.add(stampTop)
```

Tarjeta de pausa para pensar

Las demostraciones cortas se proponen como ejercicio: la tarjeta invita a pausar el vídeo, se espera unos segundos y, si se da una pista, se muestra y se vuelve a invitar a pausar. Misma tarjeta en todos los vídeos. `pista` es el texto en pantalla (puede llevar LaTeX) y `pista_voz` lo que dice la voz (por defecto, el mismo texto).

```
def pausa_para_pensar(self, pista=None, pista_voz=None, segundos=4):
    """Tarjeta que invita a pausar el vídeo; opcionalmente da una pista y vuelve a in
    def tarjeta(texto):
        barra = lambda: RoundedRectangle(corner_radius=0.06, width=0.2, height=0.6,
                                         fill_color=COLOR_ESCALAR, fill_opacity=1, st
        icono = VGroup(barra(), barra()).arrange(RIGHT, buff=0.12)
        rotulo = Tex(texto, tex_template=myTemplate, font_size=38)
        rotulo.scale_to_fit_width(min(rotulo.width, config.frame_width*0.75))
        contenido = VGroup(icono, rotulo).arrange(RIGHT, buff=0.4)
        marco = RoundedRectangle(corner_radius=0.25, width=contenido.width+0.8, heigh
                                 stroke_color=COLOR_ESCALAR, fill_color=BLACK, fill_o
        return VGroup(marco, contenido).to_edge(DOWN, buff=0.4)

    invitacion = tr("Pause el vídeo e inténtelo.", "Pause the video and give it a try
    tarjeta1 = tarjeta(invitacion)
    with self.voiceover(text=invitacion) as tracker:
        self.play(FadeIn(tarjeta1), run_time=min(0.8, tracker.duration))
    self.wait(segundos)
    if pista is not None:
        tarjeta2 = tarjeta(tr("Pista: ", "Hint: ") + pista)
        with self.voiceover(text=tr("Una pista: ", "A hint: ") + (pista_voz or pista)
            self.play(ReplacementTransform(tarjeta1, tarjeta2), run_time=min(0.8, tra
        tarjeta3 = tarjeta(tr("Pause de nuevo e inténtelo.", "Pause again and try onc
        with self.voiceover(text=tr("Pause de nuevo e inténtelo.", "Pause again and t
            self.play(ReplacementTransform(tarjeta2, tarjeta3), run_time=min(0.8, tra
        self.wait(segundos)
```

```
        tarjeta1 = tarjeta3
    self.play(FadeOut(tarjeta1), run_time=0.6)
```

Pantalla final de créditos

```
copyright = Tex(r"\textcopyright{\;} 2026 \; Marcos Bujosa")
github = Tex(r"\texttt{https://github.com/mbujosab}").next_to(copyright, DOWN)
CGG  = VGroup(copyright, github).scale(1.1)
self.add(CGG)
self.wait(15)   # tiempo para las tarjetas de enlaces de YouTube
```

3. Rodando: 1,2,3... ¡acción!

Todos los bloques reciben `idioma` (`es` o `en`) y trabajan en el repositorio correspondiente (`.` o `../videoMath2`). El manim usado es el del entorno virtual `~/.venvs/manim`.

3.1. Generamos un fichero `mpeg` por cada escena

`voz` puede ser `azure` (final) o `gtts` (pruebas gratuitas). Las escenas se indican por su número en el menú de manim (`1,2,3`) o con `a` para todas.

- Versión de poca calidad (y se abre el vídeo al terminar):

```
[ "$idioma" = "en" ] && cd ../videoMath2/lib || cd lib
echo "$escena" | VOZ=$voz ~/.venvs/manim/bin/manim -pql $fichero.py --
disable_caching
```

- Versión con calidad HD1080:

```
[ "$idioma" = "en" ] && cd ../videoMath2/lib || cd lib
echo "$escena" | VOZ=$voz ~/.venvs/manim/bin/manim -qh $fichero.py --
disable_caching
```

3.2. Montaje: concatenar las escenas, música de entrada y salida, subtítulos

El vídeo final se monta a partir de los mp4 de las escenas (no de la clase del vídeo completo): corregir una escena no obliga a renderizarlo todo ni a pagar otra vez la voz, y los límites entre escenas quedan exactos. Cada segmento se recodifica igualando la duración del audio y del vídeo (concatenar

sin recodificar deja un parón en cada unión, pues manim genera el audio unas décimas más largo que la imagen). La música suena en la portada (volumen `volumen`) y se funde cuando empieza la primera escena; vuelve al empezar la escena `cierre` (bajo la voz, con `volumen_voz`), sube en la última escena (los créditos) y se funde a negro con la imagen. La voz se normaliza con `loudnorm` y se rellena con silencio hasta la duración total (`loudnorm` retrasa unos 3 s y, sin el relleno, la mezcla acababa antes que la imagen y la música de los créditos se cortaba en seco). Los `.srt` de las escenas se desplazan y se funden en un único `video.srt` junto al mp4 (para subirlo a YouTube).

`escenas` es la lista ordenada de clases; `cierre` la escena en la que vuelve la música. El mp4 y el srt finales quedan en la raíz del repositorio del idioma.

```
export LC_ALL=C   # decimales con punto para bc y printf
[ "$idioma" = "en" ] && raiz=../videoMath2 || raiz=.
dir=$raiz/lib/media/videos/$fichero/$calidad
tmp=$raiz/tmp/$video; mkdir -p $tmp; rm -f $tmp/seg_*.mp4

# 1. Un segmento por escena, con audio y vídeo de la misma duración exacta
i=0; lista=""; offsets=""; T=0; T0=""; Tc=""
for esc in $escenas; do
  seg=$(printf "%s/seg_%02d.mp4" $tmp $i)
  dur=$(ffprobe -loglevel error -select_streams v:0 -show_entries stream=duration -
of default=nk=1:nw=1 $dir/$esc.mp4)
  fps=$(ffprobe -loglevel error -select_streams v:0 -show_entries stream=r_frame_rate
of default=nk=1:nw=1 $dir/$esc.mp4)
  if ffprobe -loglevel error -select_streams a -show_entries stream=codec_type -
of csv=p=0 $dir/$esc.mp4 | grep -q audio; then
    ffmpeg -loglevel error -y -i $dir/$esc.mp4 -af "aresample=48000,apad" -
ac 2 -t $dur -r $fps -fps_mode cfr -pix_fmt yuv420p -c:v libx264 -
crf 18 -preset fast -c:a aac -b:a 192k $seg
  else
    ffmpeg -loglevel error -y -i $dir/$esc.mp4 -f lavfi -i anullsrc=r=48000:cl=stereo
map 0:v -map 1:a -t $dur -r $fps -fps_mode cfr -pix_fmt yuv420p -
c:v libx264 -crf 18 -preset fast -c:a aac -b:a 192k $seg
  fi
  d=$(ffprobe -loglevel error -show_entries format=duration -of default=nk=1:nw=1 $se
  offsets="$offsets $T"; [ "$esc" = "$cierre" ] && Tc=$T
  printf "%2d  %-55s %8.3f s  (empieza en %8.3f)\n" $i $esc $d $T
  T=$(bc -l <<< "$T+$d"); [ -z "$T0" ] && T0=$d; dl=$d
  lista="$lista $seg"; i=$((i+1))
```

```
done
n=$i; Tl=$(bc -l <<< "$T-$dl"); [ -z "$Tc" ] && Tc=$Tl
printf "Duración total %.3f s; música hasta %.3f s, desde %.3f s (bajo la voz) y desd

# 2. Concatenación recodificada, voz normalizada, música de entrada y salida, fundido
inputs=""; fc=""; k=0
for seg in $lista; do inputs="$inputs -i $seg"; fc="$fc[$k:v][$k:a]"; k=$((k+1)); don
fin_m1=$(bc -l <<< "$T0+5"); tfade=$(bc -l <<< "$T-6")
dur_m2=$(bc -l <<< "$Tl-$Tc"); ms_m2=$(printf "%.0f" $(bc -l <<< "$Tc*1000")); fade_m
l <<< "$dur_m2-1")
fin_m3=$(bc -l <<< "$T-$Tc"); ms_m3=$(printf "%.0f" $(bc -l <<< "$Tl*1000"))
ffmpeg -loglevel error -y $inputs -i $musica -i $musica -i $musica -
filter_complex "
  ${fc}concat=n=$n:v=1:a=1[v0][a0];
  [a0]loudnorm=I=-16:LRA=11:TP=-1.5,apad=whole_dur=$T[voz];
  [$n:a]atrim=0:$fin_m1,volume=$volumen,afade=t=in:st=0:d=1,afade=t=out:st=$T0:d=5[m1
  [$((n+1)):a]atrim=0:$dur_m2,volume=$volumen_voz,afade=t=in:st=0:d=3,afade=t=out:st=
  [$((n+2)):a]atrim=$dur_m2:$fin_m3,asetpts=PTS-STARTPTS,volume=$volumen,afade=t=in:s
  [voz][m1][m2][m3]amix=inputs=4:duration=first:normalize=0[mix];
  [mix]afade=t=out:st=$tfade:d=6[a];
  [v0]fade=t=out:st=$tfade:d=6[v]" \
  -map "[v]" -map "[a]" -c:v libx264 -crf 18 -preset medium -pix_fmt yuv420p -
c:a aac -b:a 192k $raiz/$video.mp4
echo "Vídeo: $raiz/$video.mp4"

# 3. Subtítulos: desplazar el srt de cada escena y fundirlos en uno
ESCENAS="$escenas" OFFSETS="$offsets" python3 - "$dir" "$raiz/$video.srt" <<'EOF'
import sys, os, re
dir, salida = sys.argv[1], sys.argv[2]
escenas = os.environ["ESCENAS"].split(); offsets = [float(x) for x in os.environ["OFF
def t2s(t):
    h, m, s = t.strip().split(":"); s, ms = s.split(",")
    return int(h)*3600 + int(m)*60 + int(s) + int(ms)/1000
def s2t(x):
    ms = round(x*1000); h, r = divmod(ms, 3600000); m, r = divmod(r, 60000); s, ms =
    return f"{h:02d}:{m:02d}:{s:02d},{ms:03d}"
k, out = 1, []
for esc, off in zip(escenas, offsets):
    ruta = f"{dir}/{esc}.srt"
    if not os.path.exists(ruta): continue
```

```
    for bloque in re.split(r"\n\s*\n", open(ruta, encoding="utf-
8").read().strip()):
        lineas = bloque.strip().splitlines()
        if len(lineas) < 3 or "-->" not in lineas[1]: continue
        a, _, b = lineas[1].partition("-->")
        out.append(f"{k}\n{s2t(t2s(a)+off)} --> {s2t(t2s(b)+off)}\n" + "\n".join(line
open(salida, "w", encoding="utf-8").write("\n\n".join(out) + "\n")
print(f"Subtítulos: {salida} ({k-1} entradas)")
EOF
```

3.3. Comprobar las uniones y los subtítulos

Extrae un fotograma justo antes y justo después de cada instante dado (en segundos, separados por espacios; por ejemplo los .[em]pieza en"que imprime **??**) para mirar que no haya parones ni desfases.

```
[ "$idioma" = "en" ] && raiz=../videoMath2 || raiz=.
mkdir -p $raiz/tmp/fotogramas
for t in $tiempos; do
  for dt in -0.1 0.1; do
    ti=$(bc -l <<< "$t+$dt"); [ $(bc -l <<< "$ti < 0") = 1 ] && ti=0
    ffmpeg -loglevel error -y -ss $ti -i $raiz/$video.mp4 -frames:v 1 $raiz/tmp/fotog
  done
done
ls $raiz/tmp/fotogramas
```

3.4. Extraer el guion (la narración) para revisarlo

Recorre el fichero de python y saca, escena por escena, los textos `tr(...)` del idioma pedido: en prosa la narración (los que están dentro de `self.voiceover`) y entre corchetes los textos que aparecen en pantalla. Sirve para revisar el guion sin leer código y para la descripción o los subtítulos del vídeo. Como los dos py solo difieren en `LANG`, lee siempre el fichero español (`./lib/`).

```
import ast, textwrap

arbol = ast.parse(open(f"lib/{fichero}.py", encoding="utf-8").read())
idx   = 0 if idioma == "es" else 1

def texto(nodo):
    try:
```

```
        return ast.literal_eval(nodo)
    except Exception:
        return ast.unparse(nodo)

def es_tr(n):
    return (isinstance(n, ast.Call) and isinstance(n.func, ast.Name)
            and n.func.id == "tr" and len(n.args) >= 2)

def es_borrado(stmt):
    return ast.unparse(stmt).startswith("self.play(*[FadeOut(mob) for mob in self.mob

# Primera clase con construct(): la del vídeo completo.
construct = next(f for c in arbol.body if isinstance(c, ast.ClassDef)
                 for f in c.body if isinstance(f, ast.FunctionDef) and f.name == "con

escena, items = 0, []   # la portada es la escena 0
def volcar():
    global items
    if items:
        print(f"==== {'Escena' if idioma == 'es' else 'Scene'} {escena} ====\n")
        for tipo, t in items:
            if tipo == "voz":
                print(textwrap.fill(t, 72), "\n")
            else:
                print(f"    [{t}]\n")
    items = []

for stmt in construct.body:
    if es_borrado(stmt):
        volcar(); escena += 1; continue
    en_voz = set()
    for n in ast.walk(stmt):
        if isinstance(n, ast.Call) and isinstance(n.func, ast.Attribute) and n.func.a
            en_voz |= {id(s) for s in ast.walk(n) if es_tr(s)}
    for n in sorted((n for n in ast.walk(stmt) if es_tr(n)), key=lambda n: (n.lineno,
        items.append(("voz" if id(n) in en_voz else "pantalla", texto(n.args[idx])))
volcar()
```

3.5. Tangle desde la terminal (sin abrir Emacs)

```
emacs --batch -l org --eval '(org-babel-tangle-file "L01-V05-PropiedadesDeLosNumerosR
```